

FRONTEND DEVELOPER SPECIFICATION

Restaurant Reservation System

1. Project Overview

You are building the Angular 17 frontend for a Restaurant Reservation System. The backend is fully built in Spring Boot and exposes a REST API. Your job is to consume these APIs and build a clean, functional UI.

1.1 Tech Stack

Technology	Purpose
Angular 17 (Standalone)	Main frontend framework
Angular HttpClient	All API calls to the backend
Angular Router	Page navigation and route guards
Angular Reactive Forms	All forms (login, register, booking...)
localStorage	Store JWT token after login
Angular Guards (CanActivate)	Protect routes from unauthenticated access

1.2 Base URL & Headers

Every API call must use this base URL:

```
http://localhost:8081/api
```

For protected endpoints, send the JWT token in every request header:

```
Authorization: Bearer <token>
Content-Type: application/json
```

1.3 User Roles

Role	What They Can Do
CUSTOMER	Register, login, search restaurants, check availability, make reservations, view & cancel own reservations, view notifications.
ADMIN	Login, create & manage restaurants, add tables & time slots, view & confirm/cancel all reservations in their restaurant.

2. Pages & Routes

The application has 12 pages total. Build them in this order.

#	Page	Route	Access
1	Login	/login	Public
2	Register	/register	Public
3	Home (Restaurant List)	/home	Public
4	Restaurant Detail	/restaurants/:id	Public
5	Check Availability	/restaurants/:id/availability	CUSTOMER
6	My Reservations	/my-reservations	CUSTOMER
7	Notifications	/notifications	CUSTOMER
8	Admin Dashboard	/admin/dashboard	ADMIN
9	Manage Restaurant	/admin/restaurant	ADMIN
10	Manage Tables	/admin/tables	ADMIN
11	Manage Time Slots	/admin/slots	ADMIN
12	Admin Reservations	/admin/reservations	ADMIN

3. Authentication Pages

3.1 Login Page — /login

Form Fields

Field	Type	Validation
Email	email input	Required, valid email format
Password	password input	Required, min 6 characters

API Call

```
POST /api/auth/login
```

Request Body

```
{
  "email": "user@example.com",
  "password": "123456"
}
```

Response

```
{
  "token": "eyJhbGciOi...",
  "fullName": "Ahmed Ali",
  "email": "user@example.com",
  "role": "CUSTOMER"
}
```

After Login

- Save token in localStorage: `localStorage.setItem('token', response.token)`
- Save role in localStorage: `localStorage.setItem('role', response.role)`
- If role = CUSTOMER → navigate to /home
- If role = ADMIN → navigate to /admin/dashboard
- If login fails → show error message under the form

3.2 Register Page — /register

Form Fields

Field	Type	Validation
Full Name	text input	Required, min 2 characters

Field	Type	Validation
Email	email input	Required, valid email format
Password	password input	Required, min 6 characters
Phone	text input	Optional

API Call

```
POST /api/auth/register
```

Request Body

```
{
  "fullName": "Ahmed Ali",
  "email": "user@example.com",
  "password": "123456",
  "phone": "01012345678"
}
```

Response

Same as login response. Save token and navigate to /home.

Notes

- Show link: 'Already have an account? Login'
- If email already exists → backend returns 409, show error message

4. Customer Pages

4.1 Home Page — Restaurant List — /home

Purpose

Show all active restaurants. Allow filtering by city or cuisine type.

API Call

```
GET /api/restaurants
GET /api/restaurants?city=Cairo
GET /api/restaurants?cuisine=Italian
GET /api/restaurants?city=Cairo&cuisine=Italian
```

Response

```
[
  {
    "id": 1,
    "name": "La Bella Italia",
    "description": "Authentic Italian food",
    "address": "123 Tahrir St",
    "city": "Cairo",
    "phone": "0201234567",
    "cuisineType": "Italian",
    "imageUrl": "https://...",
    "openingTime": "10:00",
    "closingTime": "23:00",
    "isActive": true
  }
]
```

UI Elements

- Search bar: filter by city (text input) and cuisine (dropdown or text input)
- Restaurant cards: show image, name, city, cuisine, opening hours
- Each card has a 'View Details' button → navigate to /restaurants/:id
- Navbar: show 'Login' and 'Register' if not logged in, else show user name + logout

4.2 Restaurant Detail Page — /restaurants/:id

API Call

```
GET /api/restaurants/:id
```

UI Elements

- Show full restaurant info: name, image, description, address, phone, hours
- Show available tables list: GET /api/restaurants/:id/tables
- Button: 'Check Availability' → navigate to /restaurants/:id/availability

4.3 Check Availability Page — /restaurants/:id/availability

Form Fields

Field	Type	Validation
Date	date picker	Required, must be today or future
Guest Count	number input	Required, min 1, max 20

API Call

```
GET /api/availability?restaurantId=1&date=2025-08-15&guests=4
```

Response

```
{
  "restaurantId": 1,
  "date": "2025-08-15",
  "availableSlots": ["12:00", "14:00", "19:00"]
}
```

UI Elements

- After submitting the form, show list of available time slots as clickable buttons
- When customer clicks a slot → show 'Book Now' button
- 'Book Now' button opens a booking form below or navigates to booking step

Booking Form Fields (shown after selecting a slot)

Field	Type	Validation
Special Request	textarea	Optional, max 500 characters

(Date, guest count, and time slot are already selected — just confirm and submit)

Booking API Call

```
POST /api/reservations
Header: Authorization: Bearer <token>
```

Request Body

```
{
  "restaurantId": 1,
```

```
"reservationDate": "2025-08-15",
"timeSlotId": 3,
"guestCount": 4,
"specialRequest": "Window seat please"
}
```

Response

```
{
  "id": 10,
  "restaurantName": "La Bella Italia",
  "tableNumber": 2,
  "reservationDate": "2025-08-15",
  "slotTime": "19:00",
  "guestCount": 4,
  "status": "PENDING",
  "specialRequest": "Window seat please",
  "createdAt": "2025-08-10T14:30:00"
}
```

After Booking

- Show success message: 'Reservation created successfully!'
- Navigate to /my-reservations

4.4 My Reservations Page — /my-reservations

API Call

```
GET /api/reservations/me
Header: Authorization: Bearer <token>
```

Response

```
[
  {
    "id": 10,
    "restaurantName": "La Bella Italia",
    "tableNumber": 2,
    "reservationDate": "2025-08-15",
    "slotTime": "19:00",
    "guestCount": 4,
    "status": "PENDING",
    "specialRequest": "Window seat please",
    "createdAt": "2025-08-10T14:30:00"
  }
]
```

UI Elements

- Show reservations as cards or table rows

- Show status badge: PENDING (yellow), CONFIRMED (green), CANCELLED (red), COMPLETED (gray)
- Show 'Cancel' button only if status = PENDING or CONFIRMED

Cancel Reservation API Call

```
PUT /api/reservations/:id/cancel
Header: Authorization: Bearer <token>
```

- After cancel → refresh the list
- Show confirmation dialog before cancelling: 'Are you sure?'

4.5 Notifications Page — /notifications

API Call

```
GET /api/notifications
Header: Authorization: Bearer <token>
```

Response

```
[
  {
    "id": 1,
    "message": "Your reservation at La Bella Italia has been confirmed.",
    "type": "BOOKING_CONFIRMED",
    "isRead": false,
    "createdAt": "2025-08-10T14:31:00"
  }
]
```

Mark as Read API Call

```
PUT /api/notifications/:id/read
Header: Authorization: Bearer <token>
```

UI Elements

- List of notifications sorted by date (newest first)
- Unread notifications: highlighted with blue background or bold text
- Click on notification → mark as read (call PUT endpoint)
- Show unread count badge on the navbar notification icon

5. Admin Pages

5.1 Admin Dashboard — /admin/dashboard

UI Elements

- Welcome message with admin name
- Quick stats cards: Today's Reservations count, Total Tables, Active Time Slots
- Navigation links to: Manage Restaurant, Manage Tables, Manage Slots, View Reservations

Today's Reservations API Call

```
GET /api/reservations/restaurant/:restaurantId/today
Header: Authorization: Bearer <token>
```

5.2 Manage Restaurant — /admin/restaurant

Purpose

Admin creates or updates their restaurant profile.

Form Fields

Field	Type	Validation
Name	text input	Required
Description	textarea	Optional
Address	text input	Required
City	text input	Required
Phone	text input	Required
Cuisine Type	text input	Required
Image URL	text input	Optional
Opening Time	time input	Required (HH:mm)
Closing Time	time input	Required (HH:mm)

Create API Call

```
POST /api/restaurants
Header: Authorization: Bearer <token>
```

Request Body

```
{
```

```

    "name": "La Bella Italia",
    "description": "Authentic Italian food",
    "address": "123 Tahrir St",
    "city": "Cairo",
    "phone": "0201234567",
    "cuisineType": "Italian",
    "imageUrl": "https://...",
    "openingTime": "10:00",
    "closingTime": "23:00"
  }

```

Response

```

{ "id": 1, "name": "La Bella Italia", ... }

```

Save the restaurant id in localStorage for use in other admin pages.

5.3 Manage Tables — /admin/tables

Show existing tables

```

GET /api/restaurants/:restaurantId/tables
Header: Authorization: Bearer <token>

```

Add Table Form Fields

Field	Type	Validation
Table Number	number input	Required, min 1
Capacity	number input	Required, min 1
Location	dropdown	Required: INDOOR / OUTDOOR / TERRACE

Add Table API Call

```

POST /api/restaurants/:restaurantId/tables
Header: Authorization: Bearer <token>

```

Request Body

```

{
  "tableNumber": 1,
  "capacity": 4,
  "location": "INDOOR"
}

```

Response

```

{
  "id": 1,
  "tableNumber": 1,

```

```

    "capacity": 4,
    "location": "INDOOR",
    "isAvailable": true
  }

```

UI Elements

- Table showing all current tables with their details
- Add Table form below or in a modal
- Delete button next to each table: DELETE /api/restaurants/:restaurantId/tables/:id

5.4 Manage Time Slots — /admin/slots

Show existing slots

```

GET /api/restaurants/:restaurantId/slots
Header: Authorization: Bearer <token>

```

Add Time Slot Form Fields

Field	Type	Validation
Slot Time	time input	Required (HH:mm) e.g. 12:00, 19:30
Day of Week	dropdown	Optional: MONDAY to SUNDAY (leave empty = every day)

Add Slot API Call

```

POST /api/restaurants/:restaurantId/slots
Header: Authorization: Bearer <token>

```

Request Body

```

{
  "slotTime": "19:00",
  "dayOfWeek": "FRIDAY"
}

```

Response

```

{
  "id": 3,
  "slotTime": "19:00",
  "dayOfWeek": "FRIDAY",
  "isActive": true
}

```

5.5 Admin Reservations — /admin/reservations

Get all reservations for the restaurant

```
GET /api/reservations/restaurant/:restaurantId
Header: Authorization: Bearer <token>
```

Get today's reservations only

```
GET /api/reservations/restaurant/:restaurantId/today
Header: Authorization: Bearer <token>
```

UI Elements

- Toggle: 'All Reservations' / 'Today Only'
- Table with columns: Customer, Date, Time, Table No, Guests, Status, Actions
- Status badge: PENDING (yellow), CONFIRMED (green), CANCELLED (red)

Confirm Reservation

```
PUT /api/reservations/:id/confirm
Header: Authorization: Bearer <token>
```

- Show 'Confirm' button only if status = PENDING

Cancel Reservation

```
PUT /api/reservations/:id/cancel
Header: Authorization: Bearer <token>
```

- Show 'Cancel' button if status = PENDING or CONFIRMED

6. Auth Guard & Token Management

6.1 JWT Token Storage

- After login/register: `localStorage.setItem('token', token)`
- After login/register: `localStorage.setItem('role', role)`
- After logout: `localStorage.clear()` then navigate to `/login`

6.2 HTTP Interceptor

Create an Angular HTTP Interceptor that automatically adds the token to every request:

```
// auth.interceptor.ts
const token = localStorage.getItem('token');
if (token) {
  request = request.clone({
    headers: { Authorization: `Bearer ${token}` }
  });
}
```

6.3 Route Guards

Guard	Protects	Logic
AuthGuard	All pages except <code>/login</code> and <code>/register</code>	Check if token exists in <code>localStorage</code> . If not → redirect to <code>/login</code>
CustomerGuard	Customer pages (<code>/my-reservations</code> , <code>/notifications</code> , <code>/availability</code>)	Check <code>role = CUSTOMER</code> . If not → redirect to <code>/home</code>
AdminGuard	All <code>/admin/**</code> pages	Check <code>role = ADMIN</code> . If not → redirect to <code>/home</code>

6.4 Handle 401 Errors

If any API call returns 401 Unauthorized:

- Clear `localStorage`
- Navigate to `/login`
- Show message: 'Session expired. Please login again.'

7. Full API Reference

Base URL: <http://localhost:8081/api> | Protected endpoints need: Authorization: Bearer <token>

Method	Endpoint	Auth	Used In Page
POST	/auth/register	Public	Register page
POST	/auth/login	Public	Login page
GET	/auth/me	Any	Navbar user info
GET	/restaurants	Public	Home page
GET	/restaurants?city=X&cuisine=Y	Public	Home page filter
GET	/restaurants/:id	Public	Restaurant detail
POST	/restaurants	ADMIN	Manage restaurant
GET	/restaurants/:id/tables	Any	Restaurant detail, Admin tables
POST	/restaurants/:id/tables	ADMIN	Admin manage tables
DELETE	/restaurants/:id/tables/:tableId	ADMIN	Admin manage tables
GET	/restaurants/:id/slots	Any	Admin manage slots
POST	/restaurants/:id/slots	ADMIN	Admin manage slots
GET	/availability?restaurantId=&date=&guests=	Public	Check availability
POST	/reservations	CUSTOMER	Check availability (booking)
GET	/reservations/me	CUSTOMER	My reservations
PUT	/reservations/:id/confirm	ADMIN	Admin reservations
PUT	/reservations/:id/cancel	Any	My reservations, Admin
GET	/reservations/restaurant/:id	ADMIN	Admin reservations
GET	/reservations/restaurant/:id/today	ADMIN	Admin dashboard
GET	/notifications	CUSTOMER	Notifications page
PUT	/notifications/:id/read	CUSTOMER	Notifications page

8. Recommended Build Order

Follow this exact order to avoid blocked work:

Step	Task	Why
1	Setup Angular project + routing	Base for everything
2	Create AuthService (login, register, logout, getToken)	Needed by all pages
3	Create HTTP Interceptor	Auto-add token to all requests
4	Create Auth Guard + Admin Guard	Protect routes before building pages
5	Build Login page	Entry point for all users
6	Build Register page	Customer onboarding
7	Build Home page (restaurant list + filter)	First customer page
8	Build Restaurant Detail page	Links from Home
9	Build Check Availability + Booking page	Core customer feature
10	Build My Reservations page	Customer management
11	Build Notifications page	Customer alerts
12	Build Admin Dashboard	Admin entry point
13	Build Manage Restaurant page	Admin setup step 1
14	Build Manage Tables page	Admin setup step 2
15	Build Manage Time Slots page	Admin setup step 3
16	Build Admin Reservations page	Admin daily operations

8.1 Angular Project Setup Commands

```
ng new restaurant-frontend --standalone --routing --style=css
cd restaurant-frontend
ng generate component pages/login
ng generate component pages/register
ng generate component pages/home
ng generate service services/auth
ng generate service services/restaurant
ng generate service services/reservation
ng generate service services/notification
ng generate guard guards/auth
ng generate interceptor interceptors/auth
```